Name: Ekta Narayan Sawant

Roll no: 2401103

Batch: T22

Subject: ASD&D

Experiment 03

Aim: Multi-Service Web Application using Docker Compose

Theory:

Docker

Docker is a containerization platform used to package an application along with its dependencies into a lightweight, isolated environment called a container. Containers ensure that an application runs consistently across different systems.

Docker Image

A Docker image is a read-only template used to create containers. It contains the application code, libraries, dependencies, and configuration required to run an application.

Docker Container

A container is a running instance of a Docker image. It provides an isolated environment in which an application or service can execute.

Dockerfile

A Dockerfile is a text file containing instructions used to build a custom Docker image. In this experiment, the Dockerfile is used to create an image for the Flask backend.

Docker Compose

Docker Compose is a tool used to define and run multi-container applications. Instead of starting every container separately, all services and their configurations can be defined in a single docker-compose.yml file.

In this experiment, Docker Compose manages four services:

Service	**Purpose**
Frontend	Provides the web interface using Nginx
Backend	Runs the Flask application

PostgreSQL	Stores application data
Redis	Provides caching and maintains the visit counter

Commands used:

Sr. No.	**Command**	**Purpose**
1	mkdir docker-compose-demo	Creates the main project directory
2	cd docker-compose-demo	Navigates to the project directory
3	mkdir backend	Creates the backend directory
4	mkdir frontend	Creates the frontend directory
5	cd backend	Navigates to the backend directory
6	notepad app.py	Creates/opens the Flask application file
7	notepad requirements.txt	Creates the Python dependency file
8	notepad Dockerfile	Creates the Dockerfile for the backend
9	cd ..	Moves to the parent directory
10	cd frontend	Navigates to the frontend directory
11	notepad index.html	Creates the frontend HTML file
12	notepad docker-compose.yml	Creates/edits the Docker Compose configuration
13	notepad nginx.conf	Creates the Nginx configuration
14	dir	Displays files and folders in the current directory
15	tree /f	Displays the complete project directory structure
16	ren Dockerfile.txt Dockerfile	Renames the incorrectly created Dockerfile
17	docker compose config	Validates the Docker Compose configuration
18	docker compose build	Builds the Docker images

19	docker compose up	Creates and starts all services
20	docker compose up -d	Starts services in detached/background mode
21	docker compose ps	Displays the status of Compose services
22	docker compose down	Stops and removes the Compose containers and network
23	curl http://localhost:5000/health	Tests whether the Flask backend is accessible
24	docker compose logs	Displays service logs

Output:

docker-compose.yml:

services:

frontend:

image: nginx:alpine

ports: - "8081:80"

volumes:

- ./frontend:/usr/share/nginx/html:ro -

./nginx.conf:/etc/nginx/conf.d/default.conf:ro

depends_on:

- web

web:

build: ./backend

ports:

- "5000:5000" environment:

DB_HOST: db

```
      REDIS_HOST: redis
    depends_on:
      db:         condition:
service_healthy       redis:
        condition: service_started


  db:
    image:                    postgres:16
environment:
      POSTGRES_DB: demo_db
      POSTGRES_USER:                                    postgres
POSTGRES_PASSWORD: password
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d demo_db"]
      interval: 5s
timeout:       5s
retries: 10


  redis:
    image: redis:7


index.html:
<!DOCTYPE html>
<html>
<head>
    <title>Docker Compose Demo</title>
```

```
<style> body { font-
family: Arial, sans-serif;
background: #f2f4f7; text-
align: center; padding-top:
70px;
}

.container { background: white;
width: 500px; margin: auto; padding:
35px; border-radius: 12px; box-
shadow: 0 4px 15px rgba(0,0,0,0.15);
}
h1 {
color: #333;
}

button { padding:
12px 25px; font-size:
16px; cursor: pointer;
border: none; border-
radius: 6px;
background: #333;
color: white;
}
```

```
        #result {
margin-top: 25px;
line-height: 1.8;
        }
    </style>
</head>

<body>

<div class="container">

    <h1>Docker Compose Demo</h1>

    <p>Frontend + Flask + PostgreSQL + Redis</p>

    <button onclick="checkBackend()">
        Check Backend
    </button>

    <div id="result">
        Click the button to test the services.
    </div>
</div>

<script>        async        function
checkBackend() {
```

```
    const result = document.getElementById("result");

    result.innerHTML = "Checking services...";
     try
{

        const response = await fetch("/api");

        const data = await response.json();

        if (data.error) {              result.innerHTML =
"Error: " + data.error;
            return;
        }

        result.innerHTML =
            "Message: " + data.message + "<br>" +
            "Database: " + data.database + "<br>" +
            "Cache: " + data.cache + "<br>" +
            "Visits: " + data.visits;

    } catch (error) {

        result.innerHTML =
            "Backend is not reachable.";

    }
```

```
}
</script>

</body>
</html>
```

Requirements.txt

```
Flask psycopg2-
binary redis
```

app.py:

```
from flask import Flask, jsonify
import psycopg2 import redis
import os

app = Flask(__name__)

DB_HOST = os.getenv("DB_HOST", "db")
REDIS_HOST = os.getenv("REDIS_HOST", "redis") def get_db():
    return psycopg2.connect(
host=DB_HOST,
database="demo_db",
user="postgres",
password="password"
    )
```

```
redis_client = redis.Redis(
host=REDIS_HOST,
    port=6379,
decode_responses=True
)

@app.route("/api") def
home():
    try:
        conn = get_db()
conn.close()

        redis_client.incr("visits")
visits = redis_client.get("visits")

        return jsonify({
            "message": "Docker Compose application is working!",
            "database": "PostgreSQL connected",
            "cache": "Redis connected",
            "visits": visits
        })

    except Exception as e:
return jsonify({
            "error": str(e)
        }), 500
```

```
@app.route("/health") def
health():
    return jsonify({"status": "healthy"})



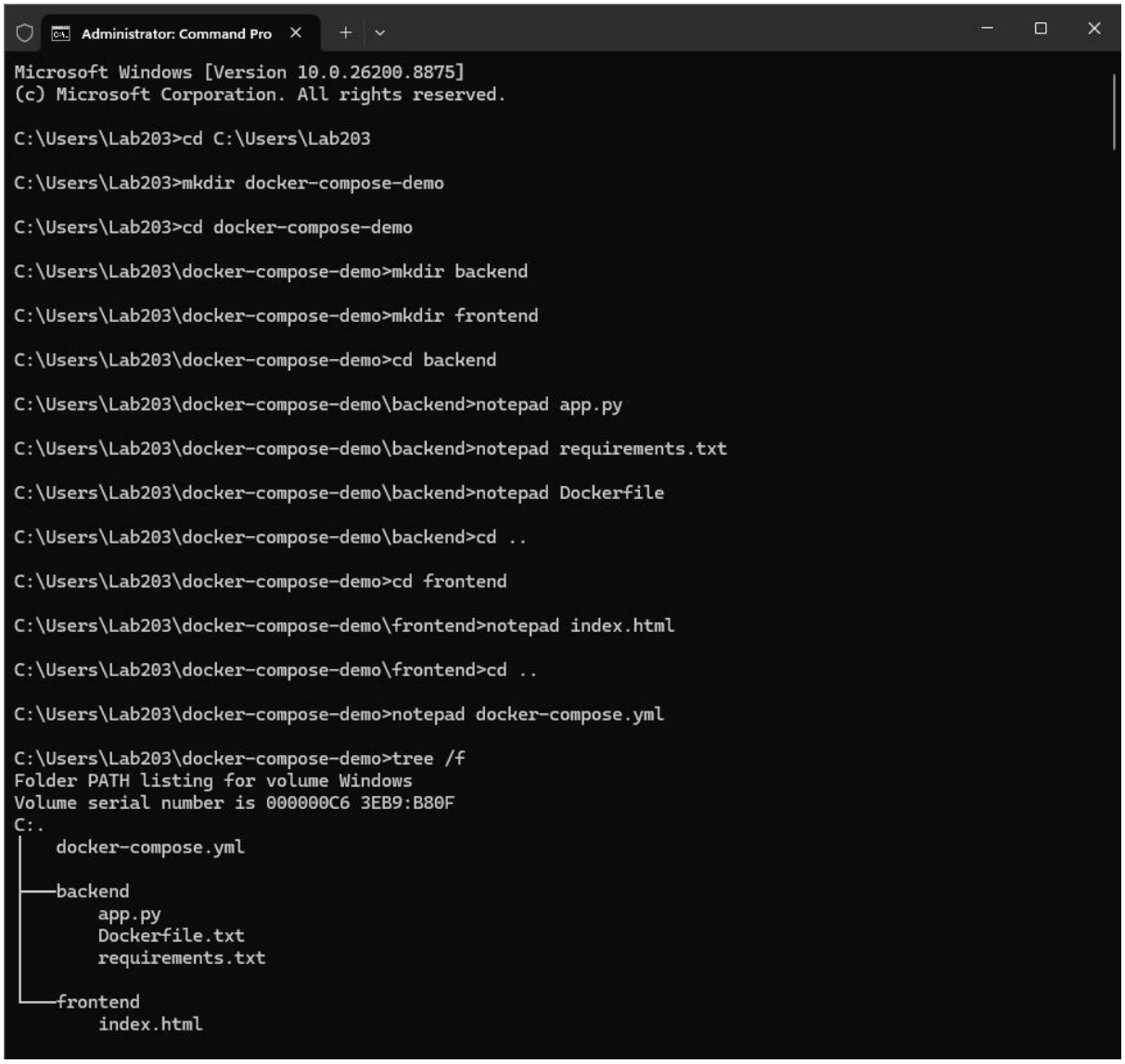
if __name__ == "__main__":
app.run(host="0.0.0.0", port=5000)

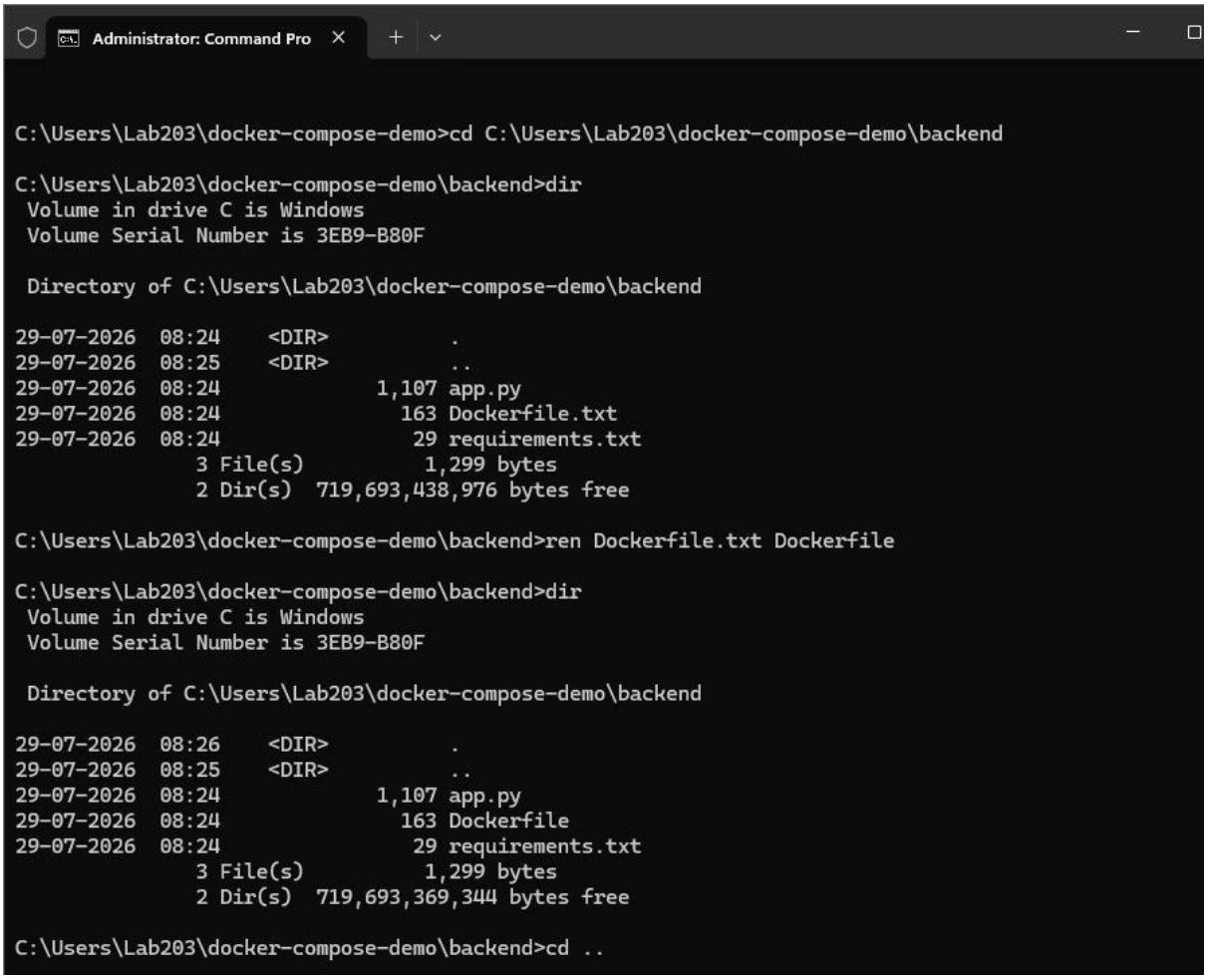
nginx.conf:
server {
listen 80;

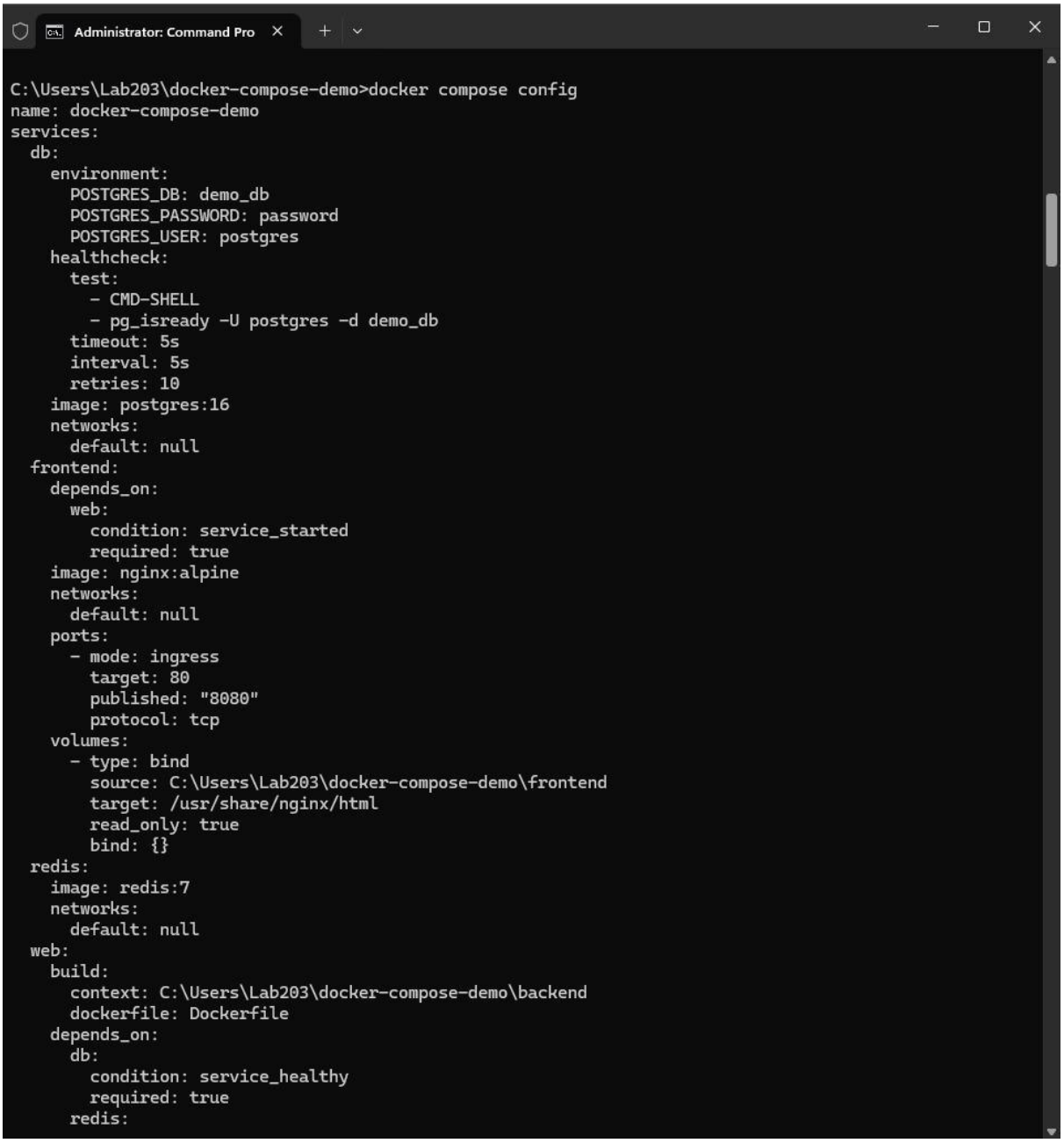
    location / { root
/usr/share/nginx/html;
index index.html;
    }

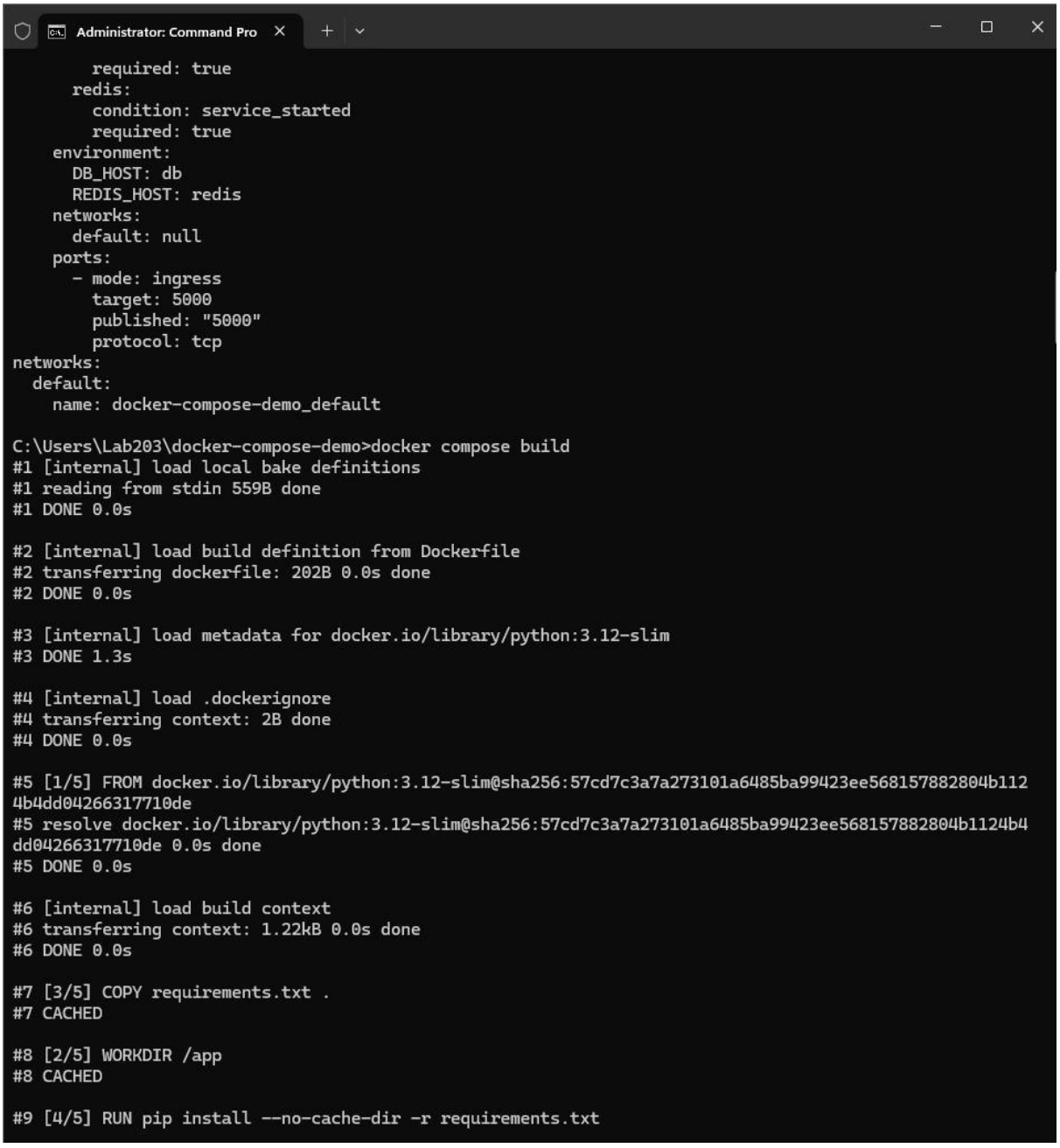
    location /api {
        proxy_pass http://web:5000;
    }

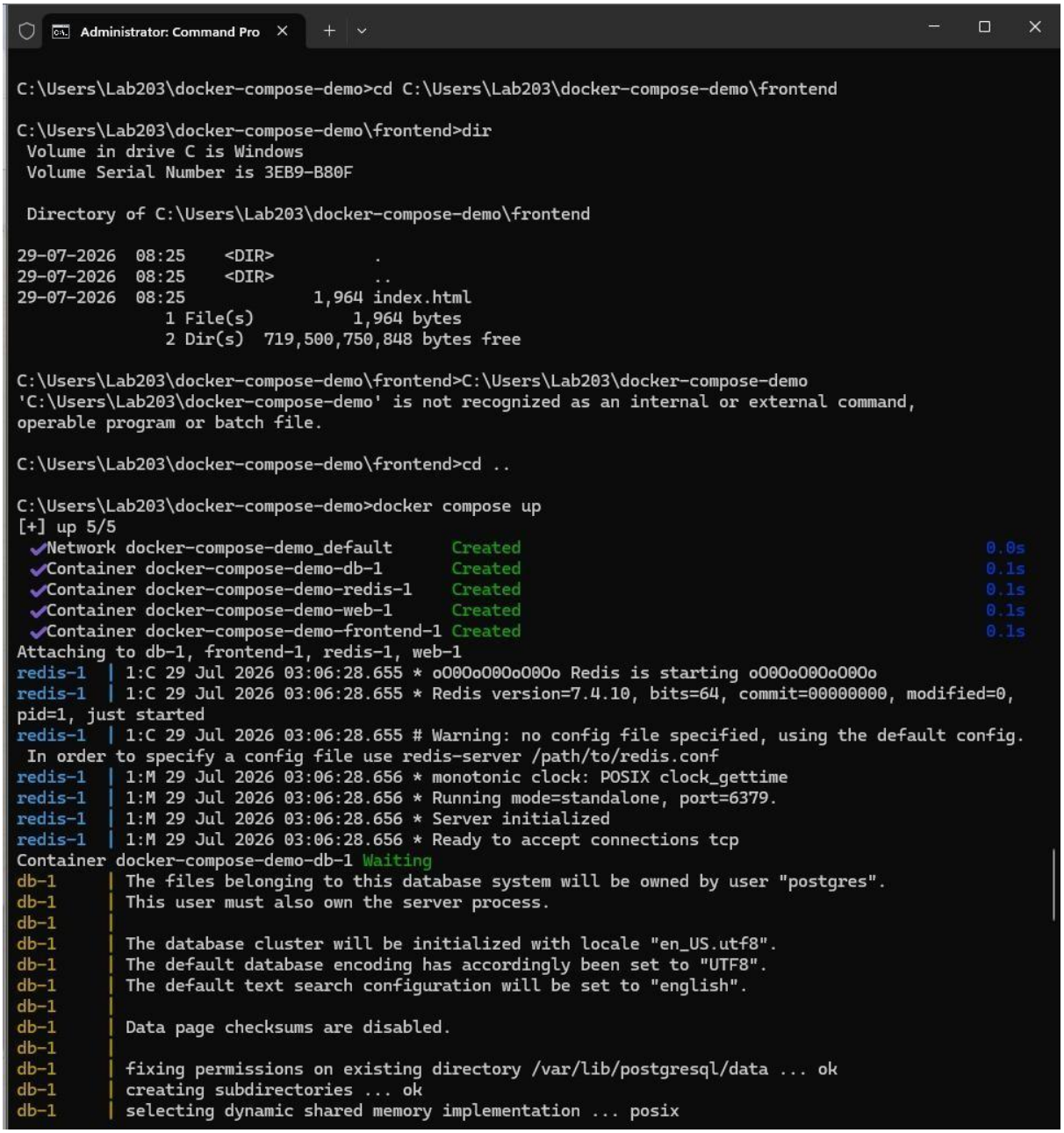
    location /health {
proxy_pass http://web:5000;
    }
}
```

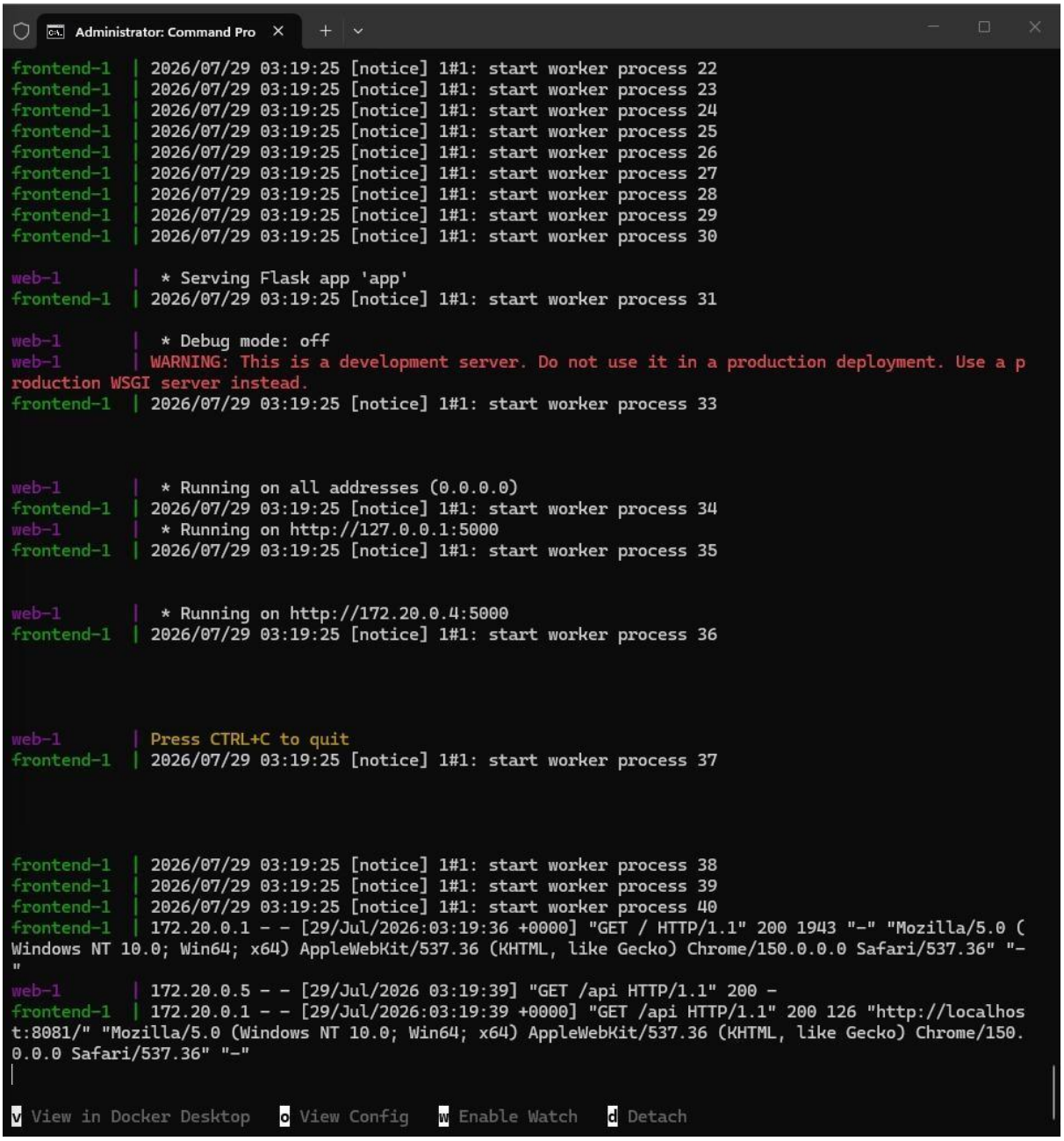

Administrator: Command Pro
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.
C:\Users\Lab203>cd C:\Users\Lab203
C:\Users\Lab203>mkdir docker-compose-demo
C:\Users\Lab203>cd docker-compose-demo
C:\Users\Lab203\docker-compose-demo>mkdir backend
C:\Users\Lab203\docker-compose-demo>mkdir frontend
C:\Users\Lab203\docker-compose-demo>cd backend
C:\Users\Lab203\docker-compose-demo\backend>notepad app.py
C:\Users\Lab203\docker-compose-demo\backend>notepad requirements.txt
C:\Users\Lab203\docker-compose-demo\backend>notepad Dockerfile
C:\Users\Lab203\docker-compose-demo\backend>cd ..
C:\Users\Lab203\docker-compose-demo>cd frontend
C:\Users\Lab203\docker-compose-demo\frontend>notepad index.html
C:\Users\Lab203\docker-compose-demo\frontend>cd ..
C:\Users\Lab203\docker-compose-demo>notepad docker-compose.yml
C:\Users\Lab203\docker-compose-demo>tree /f
Folder PATH listing for volume Windows
Volume serial number is 000000C6 3EB9:B80F
C:.
docker-compose.yml
backend
app.py
Dockerfile.txt
requirements.txt
frontend
index.html

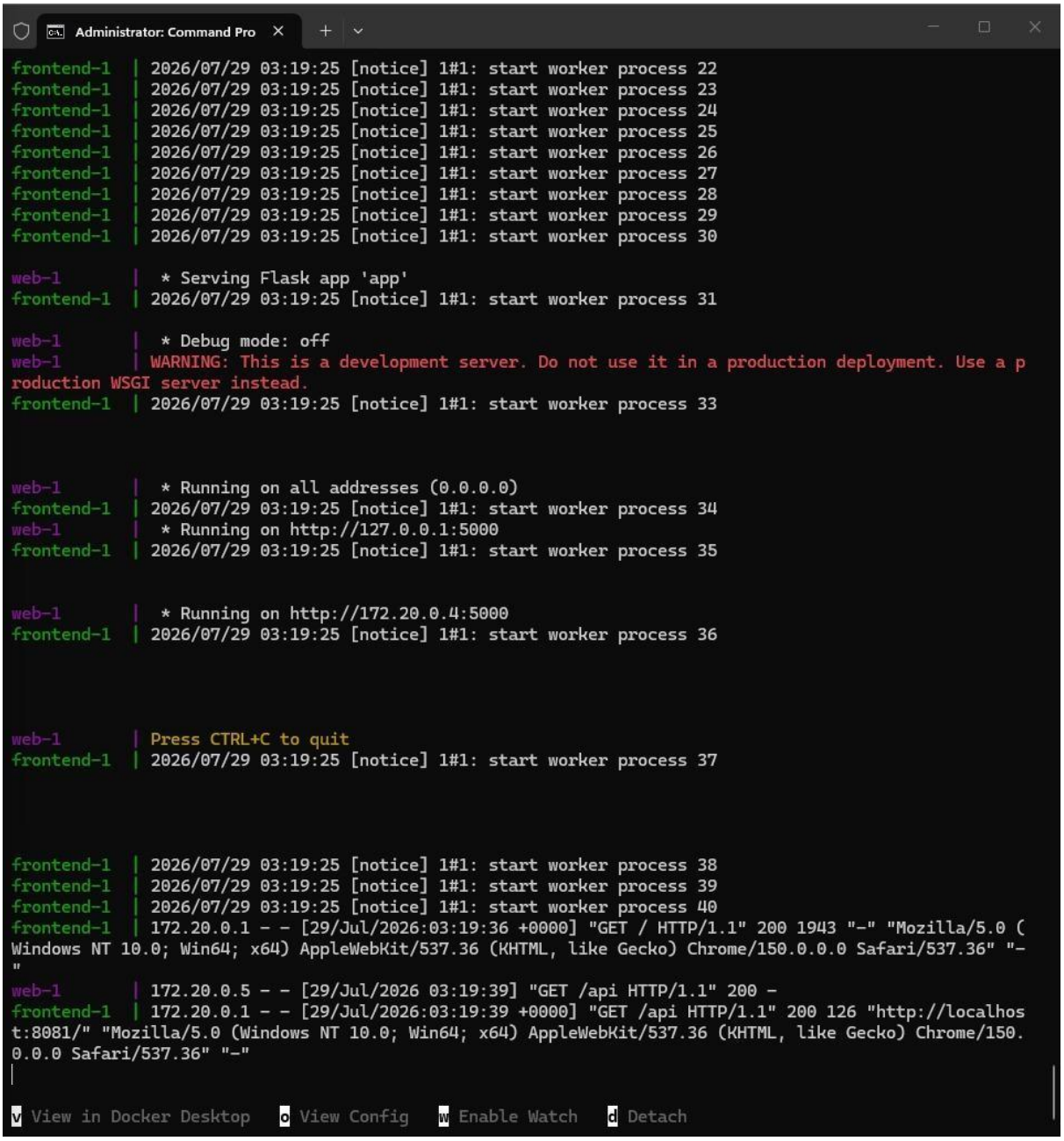

Administrator: Command Pro
C:\Users\Lab203\docker-compose-demo>cd C:\Users\Lab203\docker-compose-demo\backend
C:\Users\Lab203\docker-compose-demo\backend>dir
Volume in drive C is Windows
Volume Serial Number is 3EB9-B80F
Directory of C:\Users\Lab203\docker-compose-demo\backend
29-07-2026 08:24 <DIR> .
29-07-2026 08:25 <DIR> ..
29-07-2026 08:24 1,107 app.py
29-07-2026 08:24 163 Dockerfile.txt
29-07-2026 08:24 29 requirements.txt
3 File(s) 1,299 bytes
2 Dir(s) 719,693,438,976 bytes free
C:\Users\Lab203\docker-compose-demo\backend>ren Dockerfile.txt Dockerfile
C:\Users\Lab203\docker-compose-demo\backend>dir
Volume in drive C is Windows
Volume Serial Number is 3EB9-B80F
Directory of C:\Users\Lab203\docker-compose-demo\backend
29-07-2026 08:26 <DIR> .
29-07-2026 08:25 <DIR> ..
29-07-2026 08:24 1,107 app.py
29-07-2026 08:24 163 Dockerfile
29-07-2026 08:24 29 requirements.txt
3 File(s) 1,299 bytes
2 Dir(s) 719,693,369,344 bytes free
C:\Users\Lab203\docker-compose-demo\backend>cd ..

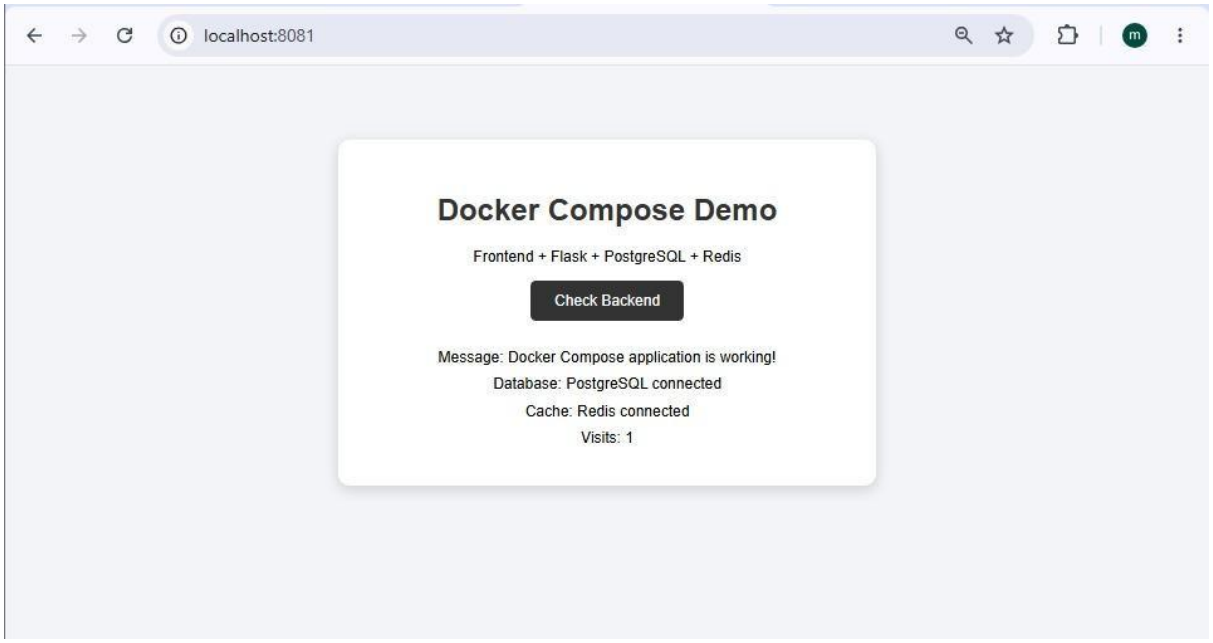

Administrator: Command Pro
C:\Users\Lab203\docker-compose-demo>docker compose config
name: docker-compose-demo
services:
db:
environment:
POSTGRES_DB: demo_db
POSTGRES_PASSWORD: password
POSTGRES_USER: postgres
healthcheck:
test:
- CMD-SHELL
- pg_isready -U postgres -d demo_db
timeout: 5s
interval: 5s
retries: 10
image: postgres:16
networks:
default: null
frontend:
depends_on:
web:
condition: service_started
required: true
image: nginx:alpine
networks:
default: null
ports:
- mode: ingress
target: 80
published: "8080"
protocol: tcp
volumes:
- type: bind
source: C:\Users\Lab203\docker-compose-demo\frontend
target: /usr/share/nginx/html
read_only: true
bind: {}
redis:
image: redis:7
networks:
default: null
web:
build:
context: C:\Users\Lab203\docker-compose-demo\backend
dockerfile: Dockerfile
depends_on:
db:
condition: service_healthy
required: true
redis:

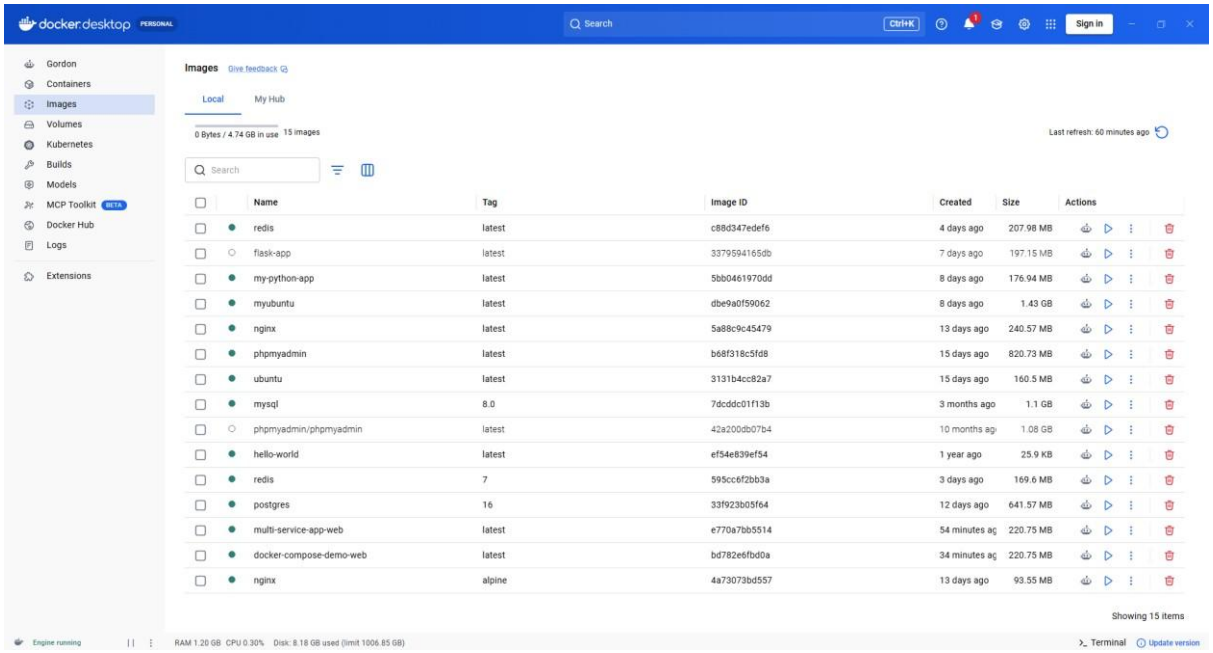

Administrator: Command Pro
required: true
redis:
condition: service_started
required: true
environment:
DB_HOST: db
REDIS_HOST: redis
networks:
default: null
ports:
- mode: ingress
target: 5000
published: "5000"
protocol: tcp
networks:
default:
name: docker-compose-demo_default
C:\Users\Lab203\docker-compose-demo>docker compose build
#1 [internal] load local bake definitions
#1 reading from stdin 559B done
#1 DONE 0.0s
#2 [internal] load build definition from Dockerfile
#2 transferring dockerfile: 202B 0.0s done
#2 DONE 0.0s
#3 [internal] load metadata for docker.io/library/python:3.12-slim
#3 DONE 1.3s
#4 [internal] load .dockerignore
#4 transferring context: 2B done
#4 DONE 0.0s
#5 [1/5] FROM docker.io/library/python:3.12-slim@sha256:57cd7c3a7a273101a6485ba99423ee568157882804b112
4b4dd04266317710de
#5 resolve docker.io/library/python:3.12-slim@sha256:57cd7c3a7a273101a6485ba99423ee568157882804b1124b4
dd04266317710de 0.0s done
#5 DONE 0.0s
#6 [internal] load build context
#6 transferring context: 1.22kB 0.0s done
#6 DONE 0.0s
#7 [3/5] COPY requirements.txt .
#7 CACHED
#8 [2/5] WORKDIR /app
#8 CACHED
#9 [4/5] RUN pip install --no-cache-dir -r requirements.txt


Administrator: Command Pro
C:\Users\Lab203\docker-compose-demo>cd C:\Users\Lab203\docker-compose-demo\frontend
C:\Users\Lab203\docker-compose-demo\frontend>dir
Volume in drive C is Windows
Volume Serial Number is 3EB9-B80F
Directory of C:\Users\Lab203\docker-compose-demo\frontend
29-07-2026 08:25 <DIR> .
29-07-2026 08:25 <DIR> ..
29-07-2026 08:25 1,964 index.html
1 File(s) 1,964 bytes
2 Dir(s) 719,500,750,848 bytes free
C:\Users\Lab203\docker-compose-demo\frontend>C:\Users\Lab203\docker-compose-demo
'C:\Users\Lab203\docker-compose-demo' is not recognized as an internal or external command,
operable program or batch file.
C:\Users\Lab203\docker-compose-demo\frontend>cd ..
C:\Users\Lab203\docker-compose-demo>docker compose up
[+] up 5/5
Network docker-compose-demo_default Created 0.0s
Container docker-compose-demo-db-1 Created 0.1s
Container docker-compose-demo-redis-1 Created 0.1s
Container docker-compose-demo-web-1 Created 0.1s
Container docker-compose-demo-frontend-1 Created 0.1s
Attaching to db-1, frontend-1, redis-1, web-1
redis-1 | 1:C 29 Jul 2026 03:06:28.655 * oO0OoO0OoO0Oo Redis is starting oO0OoO0OoO0Oo
redis-1 | 1:C 29 Jul 2026 03:06:28.655 * Redis version=7.4.10, bits=64, commit=00000000, modified=0,
pid=1, just started
redis-1 | 1:C 29 Jul 2026 03:06:28.655 # Warning: no config file specified, using the default config.
In order to specify a config file use redis-server /path/to/redis.conf
redis-1 | 1:M 29 Jul 2026 03:06:28.656 * monotonic clock: POSIX clock_gettime
redis-1 | 1:M 29 Jul 2026 03:06:28.656 * Running mode=standalone, port=6379.
redis-1 | 1:M 29 Jul 2026 03:06:28.656 * Server initialized
redis-1 | 1:M 29 Jul 2026 03:06:28.656 * Ready to accept connections tcp
Container docker-compose-demo-db-1 Waiting
db-1 | The files belonging to this database system will be owned by user "postgres".
db-1 | This user must also own the server process.
db-1 |
db-1 | The database cluster will be initialized with locale "en_US.utf8".
db-1 | The default database encoding has accordingly been set to "UTF8".
db-1 | The default text search configuration will be set to "english".
db-1 |
db-1 | Data page checksums are disabled.
db-1 |
db-1 | fixing permissions on existing directory /var/lib/postgresql/data ... ok
db-1 | creating subdirectories ... ok
db-1 | selecting dynamic shared memory implementation ... posix


Administrator: Command Pro
frontend-1 | 2026/07/29 03:19:25 [notice] 1#1: start worker process 22
frontend-1 | 2026/07/29 03:19:25 [notice] 1#1: start worker process 23
frontend-1 | 2026/07/29 03:19:25 [notice] 1#1: start worker process 24
frontend-1 | 2026/07/29 03:19:25 [notice] 1#1: start worker process 25
frontend-1 | 2026/07/29 03:19:25 [notice] 1#1: start worker process 26
frontend-1 | 2026/07/29 03:19:25 [notice] 1#1: start worker process 27
frontend-1 | 2026/07/29 03:19:25 [notice] 1#1: start worker process 28
frontend-1 | 2026/07/29 03:19:25 [notice] 1#1: start worker process 29
frontend-1 | 2026/07/29 03:19:25 [notice] 1#1: start worker process 30
web-1 | * Serving Flask app 'app'
frontend-1 | 2026/07/29 03:19:25 [notice] 1#1: start worker process 31
web-1 | * Debug mode: off
web-1 | WARNING: This is a development server. Do not use it in a production deployment. Use a p
roduction WSGI server instead.
frontend-1 | 2026/07/29 03:19:25 [notice] 1#1: start worker process 33
web-1 | * Running on all addresses (0.0.0.0)
frontend-1 | 2026/07/29 03:19:25 [notice] 1#1: start worker process 34
web-1 | * Running on http://127.0.0.1:5000
frontend-1 | 2026/07/29 03:19:25 [notice] 1#1: start worker process 35
web-1 | * Running on http://172.20.0.4:5000
frontend-1 | 2026/07/29 03:19:25 [notice] 1#1: start worker process 36
web-1 | Press CTRL+C to quit
frontend-1 | 2026/07/29 03:19:25 [notice] 1#1: start worker process 37
frontend-1 | 2026/07/29 03:19:25 [notice] 1#1: start worker process 38
frontend-1 | 2026/07/29 03:19:25 [notice] 1#1: start worker process 39
frontend-1 | 2026/07/29 03:19:25 [notice] 1#1: start worker process 40
frontend-1 | 172.20.0.1 - - [29/Jul/2026:03:19:36 +0000] "GET / HTTP/1.1" 200 1943 "-" "Mozilla/5.0 (
Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36" "-
"
web-1 | 172.20.0.5 - - [29/Jul/2026 03:19:39] "GET /api HTTP/1.1" 200 -
frontend-1 | 172.20.0.1 - - [29/Jul/2026:03:19:39 +0000] "GET /api HTTP/1.1" 200 126 "http://localhos
t:8081/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.
0.0.0 Safari/537.36" "-"
v View in Docker Desktop
o View Config
w Enable Watch
d Detach


Administrator: Command Pro
frontend-1 | 2026/07/29 03:19:25 [notice] 1#1: start worker process 22
frontend-1 | 2026/07/29 03:19:25 [notice] 1#1: start worker process 23
frontend-1 | 2026/07/29 03:19:25 [notice] 1#1: start worker process 24
frontend-1 | 2026/07/29 03:19:25 [notice] 1#1: start worker process 25
frontend-1 | 2026/07/29 03:19:25 [notice] 1#1: start worker process 26
frontend-1 | 2026/07/29 03:19:25 [notice] 1#1: start worker process 27
frontend-1 | 2026/07/29 03:19:25 [notice] 1#1: start worker process 28
frontend-1 | 2026/07/29 03:19:25 [notice] 1#1: start worker process 29
frontend-1 | 2026/07/29 03:19:25 [notice] 1#1: start worker process 30
web-1 | * Serving Flask app 'app'
frontend-1 | 2026/07/29 03:19:25 [notice] 1#1: start worker process 31
web-1 | * Debug mode: off
web-1 | WARNING: This is a development server. Do not use it in a production deployment. Use a p
roduction WSGI server instead.
frontend-1 | 2026/07/29 03:19:25 [notice] 1#1: start worker process 33
web-1 | * Running on all addresses (0.0.0.0)
frontend-1 | 2026/07/29 03:19:25 [notice] 1#1: start worker process 34
web-1 | * Running on http://127.0.0.1:5000
frontend-1 | 2026/07/29 03:19:25 [notice] 1#1: start worker process 35
web-1 | * Running on http://172.20.0.4:5000
frontend-1 | 2026/07/29 03:19:25 [notice] 1#1: start worker process 36
web-1 | Press CTRL+C to quit
frontend-1 | 2026/07/29 03:19:25 [notice] 1#1: start worker process 37
frontend-1 | 2026/07/29 03:19:25 [notice] 1#1: start worker process 38
frontend-1 | 2026/07/29 03:19:25 [notice] 1#1: start worker process 39
frontend-1 | 2026/07/29 03:19:25 [notice] 1#1: start worker process 40
frontend-1 | 172.20.0.1 - - [29/Jul/2026:03:19:36 +0000] "GET / HTTP/1.1" 200 1943 "-" "Mozilla/5.0 (
Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36" "-
"
web-1 | 172.20.0.5 - - [29/Jul/2026 03:19:39] "GET /api HTTP/1.1" 200 -
frontend-1 | 172.20.0.1 - - [29/Jul/2026:03:19:39 +0000] "GET /api HTTP/1.1" 200 126 "http://localhos
t:8081/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.
0.0.0 Safari/537.36" "-"
v View in Docker Desktop
o View Config
w Enable Watch
d Detach





Conclusion: Conclusion

From this experiment, I learned how to use Docker and Docker Compose to run a multiservice application. I learned how to create Dockerfiles, build images, and run containers. I also learned how a frontend, Flask backend, PostgreSQL database, and Redis cache can communicate with each other. I understood the use of Nginx for serving the frontend and how health checks help services start in the correct order. Overall, I learned how Docker Compose makes it easier to manage multiple services together.